HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BỘ MÔN THỰC TẬP CƠ SỞ



BÁO CÁO TIẾN ĐỘ TUẦN 3

TÍCH HỢP GEMINI VÀ ĐÁNH GIÁ ĐẦU VÀO

Đề tài: XÂY DỰNG WEBSITE HỖ TRỢ HỌC TẬP CÁ NHÂN HÓA BẰNG AI

Giảng viên hướng dẫn : **ThS.Nguyễn Xuân Đức**

Họ và tên sinh viên : **Trần Đỗ Lan Phương**

Mã sinh viên : **B23DCCN667**

Lớp : **D23CQCN09-B**

Hà Nội – 2026

I. THÔNG TIN CHUNG

Sinh viên: Trần Đỗ Lan Phương | Mã sinh viên: B23DCCN667 | Lớp: D23CQCN09-B

Giảng viên hướng dẫn: ThS. Nguyễn Xuân Đức

Tuần báo cáo: Tuần 3 — Giai đoạn Tích hợp Gemini và cài đặt chức năng Đánh giá đầu vào

Đề tài: Xây dựng website hỗ trợ học tập cá nhân hóa bằng AI.

II. MỤC TIÊU TUẦN 3

Trên nền tảng backend + auth đã hoàn thiện ở tuần 2, tuần 3 chuyển sang cài đặt phần nghiệp vụ cốt lõi — tích hợp AI và xây dựng chức năng đánh giá đầu vào:

- Tích hợp Google Gemini API qua SDK @google/genai bằng adapter pattern (interface IAIProvider) — đảm bảo có thể swap nhà cung cấp AI mà không sửa logic nghiệp vụ.
- Xây dựng QuestionGeneratorService sinh câu hỏi trắc nghiệm theo chủ đề và độ khó bằng prompt tiếng Việt, parse JSON đầu ra.
- Hoàn thiện Assessment model + controller + routes cho luồng đánh giá đầu vào (create → answer → submit → review).
- Xây dựng AssessmentGraderService dưới dạng pure functions (không DB) — phục vụ unit test dễ dàng.
- Cài đặt cơ chế phân tích kỹ năng (skill analysis) theo chủ đề và đề xuất trình độ (basic / intermediate / advanced).
- Áp dụng single in-progress guard: mỗi user chỉ có 1 bài đang dở trên mỗi môn.
- Lưu questionSnapshot bất biến trong Assessment để bảo toàn lịch sử khi câu hỏi gốc thay đổi.
- Kiểm thử end-to-end luồng đánh giá qua Swagger UI: đăng nhập → chọn môn → tạo bài → trả lời → nộp → xem kết quả.

III. KẾT QUẢ THỰC HIỆN

1 Tích hợp Google Gemini qua Adapter Pattern

Module AI được tổ chức theo Adapter pattern. Lớp AIService làm façade duy nhất cho mọi service nghiệp vụ; bên trong nó ủy quyền cho một implementation của interface

IAIProvider. Hiện tại implementation duy nhất là GeminiService, nhưng kiến trúc cho phép thêm ClaudeService, OpenAIService... mà không phải sửa các service nghiệp vụ.

Các thành phần chính:

• **IAIProvider:** Interface chung với một method generateContent(prompt) → Promise<string>.

• **GeminiService:** Implement IAIProvider; khởi tạo GoogleGenAI với GEMINI_API_KEY, dùng model "gemini-2.0-flash" (đọc từ GEMINI_MODEL).

• **AIService:** Façade — getProvider() trả về một IAIProvider; generate(prompt) gọi qua provider. Các service nghiệp vụ chỉ import aiService và gọi aiService.generate().

Lợi ích của thiết kế này:

• Tuân thủ Dependency Inversion Principle (DIP): nghiệp vụ phụ thuộc vào interface, không phụ thuộc trực tiếp vào SDK của Google.

• Dễ test: có thể thay GeminiService bằng MockAIProvider trả về JSON cố định.

• Dễ thay thế nhà cung cấp AI khi cần (đổi sang Claude, OpenAI hoặc model local).

2 Question Generator Service

QuestionGeneratorService nhận đầu vào gồm topic, difficulty (1-3), count, sau đó: (1) build prompt tiếng Việt yêu cầu Gemini sinh JSON gồm questionText, 4 options A/B/C/D, isCorrect và explanation; (2) gọi aiService.generate(prompt); (3) parse JSON, xử lý các trường hợp Gemini trả về sai format; (4) lưu Question vào collection questions để tái sử dụng cho phiên luyện tập (cơ chế bank-first ở tuần 4).

Điểm chính:

• Hàm resolveTopicNames(): map từ topicId (ObjectId) sang tên hiển thị (lấy từ Subject.topics) trước khi đưa vào prompt — giúp prompt rõ ràng hơn.

• Prompt mô tả độ khó bằng tiếng Việt (Dễ / Trung bình / Khó) và yêu cầu trả về JSON thuần (không kèm text giải thích) để dễ parse.

• Retry/fallback: nếu Gemini trả JSON sai format, log lỗi và thử lại với prompt nhắc nhở format.

• Lưu metadata.generatedBy = "gemini" để biết nguồn gốc câu hỏi.

3 Assessment Model, Controller và Routes

Module Assessment đã cài đặt đầy đủ Model + Controller + Routes. Schema Assessment tham chiếu userId + subjectId, nhúng mảng questions (questionId +

questionSnapshot + userAnswer + isCorrect). Có 2 trạng thái status: in_progress và submitted.

Các endpoint đã cài đặt (yêu cầu JWT):

• POST /api/assessments — tạo bài đánh giá mới. Kiểm tra single in-progress guard, gọi QuestionGenerator sinh 10 câu, lưu Assessment status=in_progress.

• GET /api/assessments — danh sách bài đánh giá của user (có filter theo môn, status).

• GET /api/assessments/:id — chi tiết bài (kèm câu hỏi đã sinh, đáp án người dùng nếu có).

• POST /api/assessments/:id/submit — nộp bài. Validate đáp án, gọi AssessmentGrader chấm điểm + phân tích kỹ năng, cập nhật status=submitted.

• POST /api/assessments/:id/abandon — hủy bài đang dở (giải phóng single-guard).

• DELETE /api/assessments/:id — xóa bài đánh giá khỏi lịch sử.

4 Assessment Grader Service (pure functions)

AssessmentGraderService được thiết kế dưới dạng pure functions — không truy cập database, không I/O. Các hàm chính nhận snapshot + answers, trả về kết quả đã xử lý. Nhờ đó có thể viết unit test mà không cần khởi động Express hay MongoDB.

Các hàm pure đã cài đặt:

• **gradeAnswers(snapshots, answers):** Map answers theo questionId, so sánh userAnswer với correctAnswer trong snapshot, trả về GradedQuestion[] với isCorrect. Câu không trả lời mặc định isCorrect=false (vẫn tính vào tổng).

• **buildSkillAnalysis(graded):** Bucket các câu theo topicId chính, tính { correct, total }, dùng tỉ lệ correct/total để xác định level (basic/intermediate/advanced) theo ngưỡng cố định.

• **computeScore(graded):** score = correct/total × 100 (thang 0-100).

• **suggestLevel(skillAnalysis):** Đề xuất trình độ tổng thể của user dựa trên phân bố level các chủ đề.

5 Single in-progress guard và kiểm thử end-to-end

Để đảm bảo người học không đồng thời mở nhiều bài đánh giá trên cùng một môn, hệ thống áp dụng cơ chế single in-progress guard ngay tại Controller. Khi POST /api/assessments, controller query collection assessments tìm bản ghi có { userId,

subjectId, status: in_progress }; nếu có thì trả về 409 Conflict kèm assessmentId hiện có để frontend redirect người dùng tiếp tục bài cũ.

Quy trình kiểm thử end-to-end qua Swagger UI:

- Đăng nhập tài khoản test → nhận JWT.
- POST /api/assessments với { subjectId } → nhận về assessment._id và 10 câu hỏi.
- Lặp lại POST /:id/submit với đáp án từng câu (hoặc gửi 1 lần tất cả) → nhận về score + perTopic + suggestedLevel.
- GET /:id để xem chi tiết kết quả + giải thích đáp án.
- Thử POST /api/assessments lần nữa với cùng subjectId khi chưa nộp bài → xác nhận server trả 409 Conflict (đúng).

.